

Solution Code - Microservices - Eureka_Service_Discovery

Below is the complete functional source code implementation for this assignment. All logic, validation rules, and structural requirements have been satisfied.

Source File: src\main\java\com\example\microservices\EurekaServerApplication.java

```
1 | package com.example.microservices;
2 |
3 | import org.springframework.boot.SpringApplication;
4 | import org.springframework.boot.autoconfigure.SpringBootApplication;
5 | import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;
6 |
7 | @EnableEurekaServer
8 | @SpringBootApplication
9 | public class EurekaServerApplication {
10 |     public static void main(String[] args) {
11 |         SpringApplication.run(EurekaServerApplication.class, args);
12 |         System.out.println("Eureka Service Discovery Server Initialized.");
13 |     }
14 | }
```

Source File: src\main\java\com\example\microservices\ServiceApplication.java

```
1 | package com.example.microservices;
2 |
3 | import org.springframework.boot.SpringApplication;
4 | import org.springframework.boot.autoconfigure.SpringBootApplication;
5 | import org.springframework.cloud.client.discovery.EnableDiscoveryClient;
6 |
7 | @EnableDiscoveryClient
8 | @SpringBootApplication
9 | public class ServiceApplication {
10 |     public static void main(String[] args) {
11 |         SpringApplication.run(ServiceApplication.class, args);
12 |         System.out.println("Microservice Exercise 1 running...");
13 |     }
14 | }
```

Source File: src\main\resources\application.yml

```
1 | server:
2 |     port: 8761
3 | eureka:
4 |     client:
5 |         register-with-eureka: false
6 |         fetch-registry: false
```